

MEMU - 1

E M U L A T E U R W 6 5 C 0 2

```
.=====.  
|  
|  ** MEMO-1 **                v23  
|  
| CPU   : W65C02S @ 0.75 MHz  
| RAM   : 32 Ko   (0000h - 7FFFh)  
| ROM   : 16 Ko   (C000h - FFFFh)  
| VIA   : 65C22   (son + joysticks)  
| ACIA  : 6551    (terminal Minitel 1B)  
| SON   : Buzzer piezo PWM Timer 1  
|  
| 1 - WOZMON      2 - MS-BASIC  
| 3 - CASSETTE    4 - ROM EXT  
| A - A PROPOS    R - RESTART BASIC  
|  
|_____|  
\'=====/'
```

MANUEL DE L'UTILISATEUR

< Retro Computing Project >

BipCollector & Claude // 2025

Projet Memo-1 par Benoit Aveline (Memoire Morte)

Licence Creative Commons BY-NC-SA 4.0

[PRESS RESET TO BEGIN]_

> QU'EST-CE QUE LE MEMO-1 ?

Le Memo-1 est un petit ordinateur a but pedagogique et ludique, conçu par Benoit Aveline (Memoire Morte) et publie sous licence Creative Commons BY-NC-SA 4.0. Il repose sur un processeur 65C02, la variante CMOS du legendaire MOS 6502 qui a equipe l'Apple II, le Commodore 64 ou encore la Nintendo NES.

L'originalite du Memo-1 est d'utiliser un Minitel 1B comme ecran et clavier : le micro-ordinateur n'a aucun peripherique d'affichage integre -- tout passe par la liaison serie ACIA 6551, connectee au port peri-informatique DIN-5 du Minitel.

> L'EMULATEUR MEMU-1

MEMU-1.EXE est un emulateur Windows du Memo-1 ecrit en C++17.

Il reproduit fidelement le comportement du materiel :

- * CPU W65C02S avec cycle exact (toutes instructions et modes d'adressage)
- * VIA 65C22 : Timer 1 auto-toggle pour le son, ports joystick Atari CX40
- * ACIA 6551 : liaison serie vers console Windows (remplace le Minitel)
- * RAM 32 Ko et ROM 16 Ko charges depuis memo1_rom.bin
- * ROM d'extension 8 Ko (cartouche) chargeable par glisser-deposer
- * Son via API Windows waveOut : onde carree 44100 Hz / 16 bits
- * Systeme de fichiers HDD\ pour sauvegarder / charger des .bas

> CONFIGURATION MATERIELLE EMULE

COMPOSANT	REFERENCE	DETAILS
CPU	W65C02S	0.75 MHz (25% sous-cadence volontaire)
RAM	HM62256B	32 Ko -- 0000h a 7FFFh
ROM	28C256	16 Ko -- C000h a FFFFh (memo1_rom.bin)
VIA	W65C22S	Timer 1 + ports joystick + PB7 son
ACIA	6551	Serie 19200 baud / 8N1 vers console
SON	Buzzer	Onde carree PWM via Timer 1 VIA, waveOut
EXT	28C64	ROM extension 8 Ko -- A000h a BFFFh

> CARTE MEMOIRE COMPLETE

Le W65C02 adresse 64 Ko (0000h-FFFFh). Le decodage d'adresse utilise le quartet de poids fort (A15-A12) pour decouper l'espace en blocs de 4 Ko. Vue d'ensemble :

PLAGE	TAILLE	CONTENU
0000 - 01FF	512 o	Page zero (variables BASIC) + pile 6502
0200 - 7FFF	~32 Ko	RAM utilisateur libre
8000 - 800F	16 o	VIA 65C22 (registres internes)
9000 - 9003	4 o	ACIA 6551 (liaison serie)
A000 - BFFF	8 Ko	Connecteur extension J4 (ROM cartouche)
C000 - FFFF	16 Ko	ROM systeme (WOZMON + MS-BASIC)

> REGISTRES VIA 65C22 (8000h)

Le VIA gere les ports I/O, les timers et le son.

ADRESSE	NOM	ROLE
8000h	PortB	Port B -- bit 7 (PB7) = sortie buzzer son
8001h	PortA	Port A -- joystick, lecture seulement
8002h	DDRB	Direction port B (0x80 = PB7 sortie)
8003h	DDRA	Direction port A (0x00 = entrees joystick)
8004h	T1CL	Timer 1 compteur octet bas (ecriture = latch bas)
8005h	T1CH	Timer 1 compteur haut + charge latch + demarre
8006h	T1LL	Timer 1 latch octet bas
8007h	T1LH	Timer 1 latch octet haut
8008h	ACR	Bits 7-6=11 : auto-toggle PB7 (mode TONE)

> REGISTRES ACIA 6551 (9000h)

ADRESSE	NOM	ROLE
9000h	DATA	Donnees : lecture = RX, ecriture = TX
9001h	STATUS	Etat : bit 3 = TX pret, bit 0 = RX pret
9002h	COMMAND	Mode, parite, DTR
9003h	CONTROL	Baud rate, bits de donnees, bits de stop

> INSTALLATION ET PREMIER LANCEMENT

Aucune installation n'est requise. Executable portable Windows x86-64.

- * Extraire le ZIP dans un dossier de votre choix
- * Verifier la presence de memo1_rom.bin dans le meme dossier
- * Optionnel : glisser un fichier .bin cartouche sur MEMU1.exe
- * Double-cliquer sur MEMU1.exe

[CONSOLE AU DEMARRAGE]

```
[MEMO-1] Son active (buzzer PB7).  
[MEMO-1] Ctrl+S = sauvegarder  Ctrl+O = charger  Ctrl+Q = quitter  
  
*** MEMO-1 ***  
  
1 - WOZMON   (moniteur memoire Steve Wozniak)  
2 - MS-BASIC (interpreteur Microsoft 1975)  
3 - CASSETTE (menu sauvegarde KCS)  
4 - ROM EXT  (si cartouche detectee a A000h)  
A - A PROPOS  
R - RESTART BASIC (warm start sans effacer la RAM)
```

> RACCOURCIS CLAVIER

TOUCHE	ACTION
Ctrl + Q	Quitter l'emulateur proprement (arret audio inclus)
Ctrl + S	Sauvegarder le programme BASIC courant dans HDD\
Ctrl + O	Charger un fichier .BAS depuis le dossier HDD\

> CHARGEMENT DE FICHIER BASIC (Ctrl+O)

- * Placer le fichier .bas dans HDD\ (ex : HDD\monprog.bas)
- * Dans l'emulateur, appuyer sur Ctrl+O
- * Entrer le nom du fichier sans extension puis Entree
- * Le programme est injecte ligne par ligne automatiquement
- * Attendre la fin du chargement avant de taper LIST ou RUN

! IMPORTANT : Injection ligne par ligne (500ms simulees par ligne).
Pour un fichier de 30 lignes, comptez ~15 secondes reelles.
Ne pas appuyer sur des touches pendant le chargement.

> PRESENTATION

WOZMON est le moniteur memoire ecrit par Steve Wozniak pour l'Apple 1 en 1976. 256 octets de code machine pur, loges en fin de ROM (FFxx). C'est l'outil de base pour travailler directement avec la memoire et le processeur : examiner des octets, injecter du code machine, et l'executer immediatement sans passer par un compilateur.

Penser a WOZMON comme a une "loupe sur la memoire" : vous voyez et modifiez les octets bruts tels que le CPU les lit. Tout est en HEXADECIMAL. Pas de prefixe \$ ou 0x : taper 200 ou 0200, c'est pareil.

Taper \ (backslash) pour revenir au menu de demarrage.

> COMMANDES DISPONIBLES

SYNTAXE	EFFET
AAAA	Affiche le contenu de l'octet a l'adresse AAAA
AAAA.BBBB	Dump hexadecimal de la plage AAAA a BBBB inclus
AAAA: XX YY ..	Ecrit les octets XX YY... a partir de l'adresse AAAA
AAAA R	Execute le code machine a partir de AAAA (comme JSR)
\	Retour au menu de demarrage ROM

> COMPRENDRE L'AFFICHAGE

Apres une ecriture (commande :), WOZMON reaffiche d'abord l'adresse avec son ANCIEN contenu avant de stocker les nouvelles valeurs. C'est un comportement herite de l'Apple 1, parfaitement normal.

Le curseur \ en debut de ligne signale que WOZMON attend une commande.

> EXEMPLE 1 : LIRE ET MODIFIER UN OCTET

[LECTURE / ECRITURE D'UN OCTET]

```
; On examine l'octet a l'adresse 0300
> 0300
0300: 00          ; contenu actuel = 00

; On modifie cet octet : on y met la valeur A9 (opcode LDA imm)
> 0300: A9
0300: 00          ; WOZMON reaffiche l'ANCIEN contenu avant d'ecrire

; On verifie que l'ecriture a bien eu lieu
> 0300
0300: A9          ; nouveau contenu confirme
```

> EXEMPLE 2 : DUMP D'UNE PLAGE MEMOIRE

[DUMP HEXADECIMAL]

MEMU-1 EMULATOR v23 // BipCollector & Claude // 2025 // PAGE 05 / 12

```
; Afficher 16 octets de la page zero (variables BASIC)
```

> EXEMPLE 3 : ECRIRE ET EXECUTER UN PROGRAMME

On va ecrire un mini-programme de 6 octets qui envoie le caractere 'A' (code ASCII 0x41) sur le port serie (ACIA a 9000h) puis revient.
Les instructions 6502 en assembleur et leur equivalent hexa :

```
[ PROGRAMME : ENVOYER UN CARACTERE SUR SERIE ]

; LDA #$41 --> A9 41   (charger la valeur 0x41 dans A)
; STA $9000 --> 8D 00 90 (ecrire A dans le registre TX de l'ACIA)
; RTS          --> 60    (retourner a WOZMON)

; Depot des 6 octets a partir de 0200 :
> 0200: A9 41 8D 00 90 60
0200: 00          ; ancien contenu (normal)

; Verification de la plage deposee :
> 0200.0205
0200: A9 41 8D 00 90 60

; Execution du programme :
> 0200 R
A          ; le caractere 'A' s'affiche -> programme OK
```

> EXEMPLE 4 : BOUCLE QUI COMPTE EN MEMOIRE

Un programme qui incremente l'octet a l'adresse 00F0 en boucle.
(Ctrl+C ou RESET pour arreter.)

```
[ PROGRAMME : COMPTEUR EN MEMOIRE ]

; INC $F0 --> E6 F0   (incrementer l'octet a l'adresse 00F0)
; JMP $0200 --> 4C 00 02 (boucler sur soi-meme)

> 0200: E6 F0 4C 00 02
> 0200 R          ; lancer la boucle

; Appuyer RESET, puis verifier :
> 00F0          ; la valeur a change au fil des cycles
00F0: 7A          ; exemple : 122 passages en boucle
```

> EXEMPLE 5 : LIRE LA ROM SYSTEME

Les vecteurs 6502 (NMI, RESET, IRQ) sont toujours aux adresses FFFA-FFFF. On peut les lire avec WOZMON pour savoir ou pointe le code de demarrage de la ROM :

```
[ LECTURE DES VECTEURS 6502 ]

> FFFA.FFFF
MEMU:1 EMULATOR v23 // BinCollector & Claude // 2025 // PAGE 06 / 12
FFFA: 00 0F      ; vecteur NMI -> 0F00h
FFFC: DA E1      ; vecteur RESET -> E1DAh (little-endian)
FFFF: FB E3      ; vecteur IRQ -> E3FBh
```

> PRESENTATION ET HISTORIQUE

L'interpreteur MS-BASIC embarque dans la ROM du Memo-1 est la lignee directe du premier produit commercial de Microsoft : un interpreteur BASIC ecrit en 1975 par Bill Gates, Paul Allen et Monte Davidoff pour l'Altair 8800. Le port 6502 (Ric Weiland) tourne ici cinquante ans plus tard, desassemble et restaure par Michael Steil.

Le BASIC est un langage INTERPRETE : chaque ligne est lue et executee a la volee, a chaque lancement. C'est plus lent que de l'assembleur compile, mais on peut ecrire une ligne, taper RUN, et voir le resultat immediatement, sans etape de compilation. Ideal pour apprendre.

Chaque instruction commence par un NUMERO DE LIGNE (10, 20, 30...). Par convention on saute de 10 en 10 pour pouvoir inserer des lignes. Le programme s'execute dans l'ordre croissant des numeros.

> COMMANDES DE BASE

COMMANDE	EFFET
LIST	Affiche le programme numerote en memoire
LIST 10-50	Affiche seulement les lignes 10 a 50
RUN	Execute depuis la premiere ligne
RUN 50	Execute a partir de la ligne 50
NEW	Efface le programme et les variables
CONT	Reprend apres STOP ou interruption
PRINT expr	Affiche valeur ou texte. ; = colle, , = tabule
INPUT "msg";v	Affiche msg et attend une saisie dans v
GET a\$	Lit un caractere sans attendre Entree
LET v = expr	Affecte une valeur (LET optionnel : v = expr suffit)
REM texte	Commentaire, ignore a l'execution
END	Termine le programme
STOP	Interrompt (reprise avec CONT)

> STRUCTURES DE CONTROLE

STRUCTURE	DESCRIPTION
FOR i=1 TO n	Debut de boucle. i prend les valeurs 1,2,...,n
FOR i=1 TO n STEP s	Boucle avec pas s (negatif pour descendre)
NEXT i	Fin de boucle FOR
IF cond THEN instr	Execute instr si cond est vraie
IF cond THEN n	Saute a la ligne n si cond est vraie
GOTO n	Saut inconditionnel a la ligne n
GOSUB n	Appel de sous-programme a la ligne n
RETURN	Retour depuis un sous-programme
ON x GOTO a,b,c	Saute a la a-ieme ligne selon valeur de x
ON x GOSUB a,b,c	Idem mais en sous-programme
WHILE cond	Boucle tant que cond est vraie
WEND	Fin de boucle WHILE

> VARIABLES ET TYPES

Pas de declaration obligatoire : les variables existent des l'utilisation.

Une lettre ou une lettre + chiffre : A, B, X1, Y2... = nombre reel.

Nom termine par \$: chaine de caracteres. NOM\$, A\$, R2\$...

Nom termine par % : entier (plus rapide). I%, J%, K%...

> FONCTIONS DISPONIBLES

FONCTION	DESCRIPTION
ABS(x)	Valeur absolue
INT(x)	Partie entiere (arrondi vers le bas)
RND(x)	Nombre aleatoire entre 0 et 1
SGN(x)	Signe : -1, 0 ou 1
SQR(x)	Racine carree
LOG(x)	Logarithme naturel (base e)
EXP(x)	Exponentielle e^x
SIN(x)/COS(x)	Fonctions trigonometriques (radians)
ASC(a\$)	Code ASCII du premier caractere de a\$
CHR\$(n)	Caractere correspondant au code ASCII n
LEN(a\$)	Longueur d'une chaine
LEFT\$(a\$,n)	n premiers caracteres de a\$
RIGHT\$(a\$,n)	n derniers caracteres de a\$
MID\$(a\$,p,n)	n caracteres a partir de la position p
STR\$(x)	Convertit un nombre en chaine
VAL(a\$)	Convertit une chaine en nombre
PEEK(n)	Lit l'octet en memoire a l'adresse n
FRE(0)	Memoire libre restante en octets
JOY(n)	Etat joystick n (0 ou 1) sur 5 bits (Memo-1)

> EXEMPLE 1 : BONJOUR ET BOUCLE

Le programme classique pour debuter : afficher un message en boucle.
Ctrl+C ou RESET pour arreter.

[BONJOUR ET BOUCLE FOR]

```

10 REM PROGRAMME BONJOUR - MEMO-1
20 PRINT "BONJOUR DEPUIS LE MEMO-1 !"
30 GOTO 20
RUN

; Variante avec compteur :
10 FOR I = 1 TO 10
20   PRINT "LIGNE NUMERO "; I
30 NEXT I
40 PRINT "FIN DU PROGRAMME"
50 END

```

> EXEMPLE 2 : SAISIE ET CALCUL

[SAISIE ET CALCULS]

```

10 INPUT "ENTRER UN NOMBRE : "; N
20 IF N <= 0 THEN PRINT "NOMBRE POSITIF SVP" : GOTO 10
30 PRINT "RACINE CARREE = "; SQR(N)
40 PRINT "DOUBLE      = "; N * 2
50 PRINT "CARRE       = "; N ^ 2
60 PRINT "RECOMMENCER ? (O/N)"
70 GET A$
80 IF A$ = "O" THEN GOTO 10
90 END

```

> EXEMPLE 3 : SOUS-PROGRAMMES ET TABLEAUX

[TABLEAUX ET SOUS-PROGRAMMES]

```

10 DIM A(10)
20 FOR I = 1 TO 10
30   A(I) = INT(RND(1) * 100)
40 NEXT I
50 GOSUB 200      ; affichage du tableau
60 GOSUB 300      ; calcul de la somme
70 END

200 REM -- SOUS-PROG AFFICHAGE --
210 FOR I = 1 TO 10 : PRINT A(I); " "; : NEXT I
220 PRINT "" : RETURN

300 REM -- SOUS-PROG SOMME --
310 S = 0
320 FOR I = 1 TO 10 : S = S + A(I) : NEXT I
330 PRINT "SOMME = "; S; " MOYENNE = "; S/10
340 RETURN

```

> PRINCIPE PHYSIQUE DU SON

Le Memo-1 n'a pas de puce sonore dediee. Le son est produit par la broche PB7 du VIA 65C22, reliee directement a un buzzer piezo passif.

La ROM active le mode "free-run auto-toggle" du Timer 1 en ecrivant 0xC0 dans le registre ACR (bits 7-6 = 11). Dans ce mode, le Timer 1 decremente depuis son latch (T1LH:T1LL) et bascule PB7 a chaque underflow, sans intervention du CPU. On obtient une onde carree :

[FORMULE TONE @ 0.75 MHz]

```
freq (Hz)  = CPU_Hz / (valeur_TONE * 2)
valeur     = CPU_Hz / (freq * 2)
```

```
A 0.75 MHz : valeur = 750000 / (freq * 2)
LA 440 Hz  : 750000 / 880  = 852
DO 261 Hz  : 750000 / 522  = 1433
```

> TABLE DES NOTES -- CPU 0.75 MHz

NOTE	FREQUENCE	VALEUR	EXEMPLE
DO	261.63 Hz	1433	TONE 1433 ; Do central C4
DO#	277.18 Hz	1353	TONE 1353
RE	293.66 Hz	1277	TONE 1277
RE#	311.13 Hz	1205	TONE 1205
MI	329.63 Hz	1138	TONE 1138
FA	349.23 Hz	1074	TONE 1074
FA#	369.99 Hz	1014	TONE 1014
SOL	392.00 Hz	957	TONE 957
SOL#	415.30 Hz	903	TONE 903
LA	440.00 Hz	852	TONE 852 ; Diapason standard
LA#	466.16 Hz	804	TONE 804
SI	493.88 Hz	759	TONE 759

> MELODIE DO-MI-SOL EN BASIC

[MELODIE BASIC]

```
10 TONE 1433 : FOR I=0 TO 120:NEXT I ; DO
20 TONE 0    : FOR I=0 TO 30:NEXT I
30 TONE 1138 : FOR I=0 TO 80:NEXT I ; MI
40 TONE 0    : FOR I=0 TO 30:NEXT I
50 TONE 957  : FOR I=0 TO 80:NEXT I ; SOL
60 TONE 0    : FOR I=0 TO 30:NEXT I
70 TONE 1433 : FOR I=0 TO 160:NEXT I ; DO (long)
80 TONE 0 : END
```

> PROBLEMES COURANTS ET SOLUTIONS

SYMPTOME	SOLUTION
Ecran noir, pas de menu	Verifier memo1_rom.bin present (meme dossier)
Craquements continus	Bug latch T1LH -- recompiler v22+ corrige
Pas de melodie au boot	Meme cause -- VIA6522.cpp write(T1CH) fix
Ctrl+0 sans effet	Fichier doit etre dans HDD\ pas a la racine
Lignes superposees au LOAD	Corrige v23 : injection une ligne a la fois
Son faux (mauvaise freq)	CPU a 0.75MHz : utiliser table page 10
TONE sans effet	Verifier ACR VIA -- doit recevoir 0xC0
ROM extension ignoree	ROM non programmee ou connecteur sens inverse
Joystick ne repond pas	Verifier DDRA=0x00 et connexion J1/J2
Caracteres corrompus	Minitel 1 (pas 1B) incompatible -- verifier
BASIC plante (?) au RUN	RAM corrompue -- taper NEW puis recharger
OUT OF MEMORY	Programme trop long -- utiliser FRE(0) pour verifier

> HISTORIQUE DES VERSIONS EMULATEUR

VER.	MODIFICATION
v21	Son implemente : Timer 1 auto-toggle + AudioEngine waveOut 44100Hz
v22	Fix latch T1LH : write(T1CH) met a jour regs[7] et regs[6]
v22	Fix tick VIA synchronise cycle par cycle dans la boucle CPU
v22	Fix silence demarrage : AudioEngine emet 0 si ACR != mode toggle
v23	CPU 0.75 MHz (-25%) -- valeurs TONE recalculées (750000/freq/2)
v23	Fix Ctrl+0 : injection ligne par ligne, 500k cycles entre chaque
v23	Ajout HDD\test_tone.bas : 12 demi-tons a 0.75 MHz

> COMMANDES UTILES EN CAS DE PROBLEME

[DIAGNOSTICS]

```

; Depuis WOZMON : verifier la RAM (doit etre lisible/modifiable)
> 0300: AA BB CC DD
> 0300.0303
0300: AA BB CC DD ; si different -> probleme materiel/emulateur

; Depuis BASIC : afficher la memoire libre
PRINT FRE(0)

; Depuis BASIC : lire un octet VIA (port B)
PRINT PEEK(32768) ; 32768 = 8000h

```

> MATERIEL ORIGINAL

Benoit Aveline (Memoire Morte)
<https://github.com/MemoireMorte/Memo-1>
Licence Creative Commons BY-NC-SA 4.0
Gerbers, BOM, code source, schematiques disponibles sur GitHub

> EMULATEUR

BipCollector & Claude -- Retro Computing Project -- 2025
Ecrit en C++17, compile avec MinGW-w64 (GCC 13)
Audio : API Windows waveOut, 44100 Hz, 16 bits, 4 buffers
Sources incluses dans le ZIP (dossier SRC\)

> LOGICIELS EMBARQUES EN ROM

WOZMON
Steve Wozniak -- Apple 1, 1976
Moniteur memoire 256 octets, domaine public

MS-BASIC (BASIC-M6502)
Bill Gates, Paul Allen, Monte Davidoff -- Microsoft, 1975
Port 6502 : Ric Weiland
Desassemblage et restauration : Michael Steil (pagetable.com)

ca65 / cc65
Assembleur/compilateur 6502 utilise pour la ROM Memo-1
<https://cc65.github.io>

> INSPIRATIONS ET REFERENCES

Ben Eater
Serie YouTube "Build a 6502 computer from scratch" (CC-BY)
<https://eater.net/6502>

Ian Ward
Tutoriels LCD 2004 et timer 555

6502.org
Reference opcodes, modes d'adressage et datasheets W65C02S
<https://www.6502.org>

Western Design Center (WDC)
Datasheets W65C02S, W65C22S
<https://www.westerndesigncenter.com>

*** FIN DU MANUEL ***

BipCollector & Claude // Retro Computing Project // 2025